



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Segurança de Sistemas Informáticos

Trabalho Prático

Hélder Cruz

a104174

André Pinto

a104267

Rui Amaral

a104452

0. Índice

1	Resumo	1
2	Introdução	2
2.1	Enquadramento do projeto	2
2.2	Objetivos da solução	2
2.3	Visão geral das funcionalidades implementadas	2
3	Arquitetura da Solução	4
3.1	Arquitetura funcional	4
3.2	Organização do projeto	4
3.3	Protocolo de comunicação	5
3.4	Persistência de dados	5
3.5	Decisões arquiteturais principais	5
4	Funcionalidades e Fluxos de Comunicação	7
4.1	Registo e autenticação	7
4.2	Gestão de contactos e utilizadores online	7
4.3	Envio e receção de mensagens	7
4.4	Mensagens offline e confirmação de entrega	8
4.5	Sessões criptográficas e rekey	8
5	Modelo de Segurança e Gestão de Chaves	9
5.1	Modelo de ameaça	9
5.2	Servidor honesto mas curioso	9
5.3	Atacante de rede e proteção contra MITM	9
5.4	Identidade dos utilizadores	9
5.5	Gestão de chaves no cliente e no servidor	10
5.6	Confiança, TOFU e limitações de identidade	10
6	Primitivas Criptográficas e Garantias	11
6.1	Ed25519	11
6.2	X25519 e HKDF-SHA256	11
6.3	ChaCha20-Poly1305	11
6.4	TLS	11
6.5	Garantias de segurança	11
7	Valorizações e Hardening	13
7.1	Mensagens offline	13
7.2	Ack de entrega	13
7.3	Sessões com rekey e reply prekeys	14
7.4	Validação de inputs e controlo de acesso	15
7.5	Proteção de ficheiros sensíveis	15
7.6	Smoke tests realizados	15
8	Limitações e Trabalho Futuro	16
8.1	Limitações da solução	16
8.2	Melhorias futuras	16
9	Conclusão	17

0. Lista de Figuras

Figura 1	Arquitetura funcional da solução.	3
Figura 2	Fluxo de Mensagens Offline.	13
Figura 3	Fluxo de Ack e Inbox.	14
Figura 4	Sessões criptográficas, rekey e reply prekeys	14

1. Resumo

Este projeto foi desenvolvido no contexto da unidade curricular de Segurança de Sistemas Informáticos e consistiu na implementação de um sistema de conversação seguro com End-to-End Encryption (E2EE).

A solução segue uma arquitetura cliente-servidor em Python. O servidor é responsável pela coordenação do sistema, incluindo gestão de utilizadores, contactos, encaminhamento e armazenamento de mensagens offline. No entanto, o conteúdo das mensagens é cifrado no cliente remetente e apenas decifrado no cliente destinatário, impedindo que o servidor aceda ao plaintext.

Foram implementados mecanismos de autenticação baseada em challenge-response com Ed25519, comunicação cliente-servidor protegida por TLS com TOFU e certificate pinning, cifragem ponta-a-ponta com X25519, HKDF-SHA256 e ChaCha20-Poly1305, assinatura de envelopes com Ed25519, suporte a mensagens offline com acknowledgements de entrega e uma valorização intermédia de forward secrecy através de sessões criptográficas com rekey.

Além dos mecanismos criptográficos, foi realizada uma fase de hardening da implementação, incluindo validação de inputs, controlo de acesso no servidor, limites de payload, permissões restritas para ficheiros sensíveis, escrita atómica e smoke tests. A solução apresenta ainda algumas limitações, nomeadamente a ausência de uma PKI completa para utilizadores, a dependência do servidor para descoberta de chaves públicas e a inexistência de um protocolo double ratchet completo.

2. Introdução

2.1. Enquadramento do projeto

Este projeto foi desenvolvido no contexto da unidade curricular de Segurança de Sistemas Informáticos e tem como objetivo a construção de um sistema de conversação seguro entre utilizadores.

A solução desenvolvida segue uma arquitetura cliente-servidor em Python, na qual o servidor coordena utilizadores, contactos, encaminhamento e armazenamento de mensagens, mas não possui as chaves necessárias para decifrar o conteúdo das conversas.

Num sistema de chat tradicional, o servidor recebe e armazena mensagens em claro, tornando-se um ponto crítico de confiança. Neste projeto, optámos por aplicar End-to-End Encryption (E2EE), garantindo que as mensagens são cifradas no cliente remetente e apenas decifradas no cliente destinatário.

Além da confidencialidade garantida pela E2EE, a solução considera também integridade, autenticidade e proteção contra atacantes de rede. Para isso, o canal cliente-servidor foi protegido com TLS, recorrendo a uma identidade persistente do servidor e a um mecanismo de Trust On First Use (TOFU) com pinning local.

2.2. Objetivos da solução

O objetivo principal é criar um sistema de mensagens seguro onde o servidor funciona como intermediário funcional, transportando e armazenando informação sem aceder ao seu conteúdo.

Em resumo, os principais objetivos foram:

- garantir confidencialidade, integridade e autenticidade das mensagens;
- impedir que o servidor leia o conteúdo das comunicações;
- autenticar utilizadores sem recurso a passwords armazenadas no servidor;
- proteger a ligação cliente-servidor contra ataques de rede;
- suportar mensagens offline através de armazenamento cifrado;
- implementar confirmação de entrega com estados `pending` e `delivered`;
- incluir uma valorização intermédia de forward secrecy com sessões e rekey;
- aplicar medidas de hardening, como validação de inputs, controlo de acesso e permissões restritas para ficheiros sensíveis.

A solução não pretende implementar uma PKI completa nem reproduzir protocolos complexos como o Signal. Essas opções são assumidas como limitações e possíveis melhorias futuras.

2.3. Visão geral das funcionalidades implementadas

A solução é composta por um servidor TCP e um cliente de linha de comandos, que comunicam através de um protocolo próprio baseado em JSON. O cliente permite realizar operações como registo, login, gestão de contactos, envio de mensagens, consulta da inbox e encerramento da sessão.

A autenticação é feita através de challenge-response com Ed25519. O servidor gera um desafio aleatório e o cliente prova a posse da sua chave privada assinando esse desafio. Desta forma, evita-se o armazenamento de passwords no servidor.

As mensagens são protegidas ponta-a-ponta com X25519, HKDF-SHA256 e ChaCha20-Poly1305. Os envelopes das mensagens são assinados digitalmente com Ed25519, permitindo validar a origem e integridade dos dados recebidos.

O sistema suporta mensagens offline. O servidor guarda envelopes cifrados com estado **pending**, e o cliente destinatário envia um acknowledgement após conseguir processar e validar a mensagem, permitindo ao servidor marcá-la como **delivered**.

Como valorização adicional, foi implementado um mecanismo de sessões criptográficas por direção, com rekey automático e reply prekeys. Esta abordagem melhora parcialmente as propriedades de forward secrecy, sem introduzir a complexidade de um double ratchet completo.

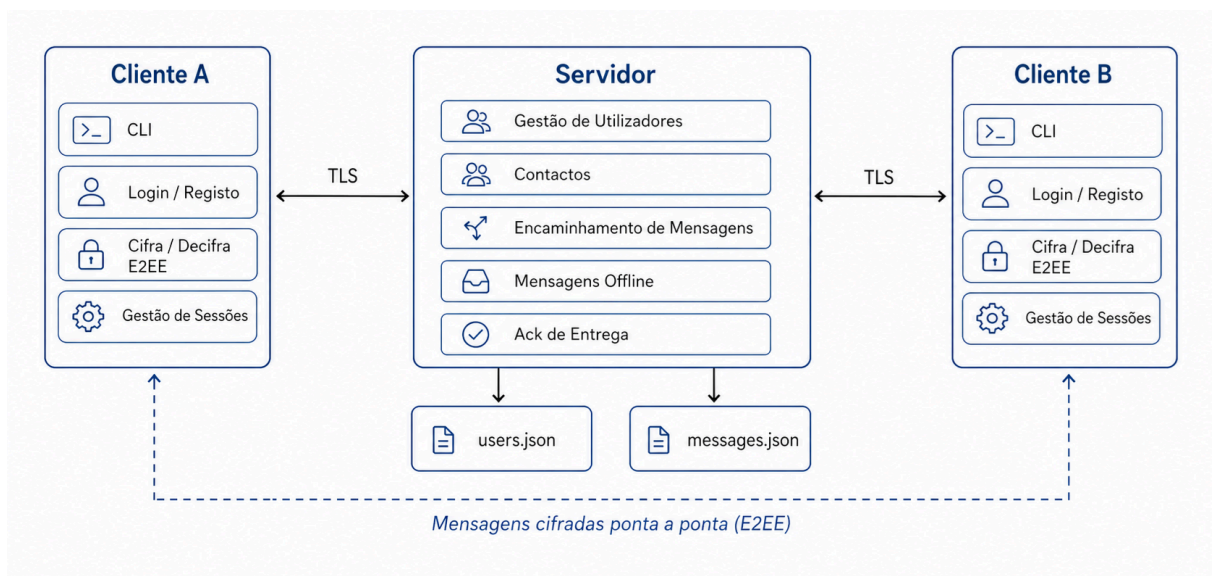


Figura 1: Arquitetura funcional da solução.

3. Arquitetura da Solução

3.1. Arquitetura funcional

A solução segue um modelo cliente-servidor. O servidor atua como ponto central de coordenação, enquanto cada utilizador interage com o sistema através de uma instância do cliente.

Optámos por este modelo porque simplifica a coordenação entre utilizadores, permite centralizar a presença online e facilita o suporte a mensagens offline. Esta escolha também está alinhada com o enunciado do projeto, que prevê a existência de um servidor responsável pela gestão de utilizadores, encaminhamento de mensagens e armazenamento de metadados necessários ao funcionamento do sistema.

Ao mesmo tempo, as operações criptográficas sensíveis são mantidas no cliente. O servidor não participa na decifragem do conteúdo das mensagens nem possui as chaves privadas dos utilizadores. Esta separação permite manter a arquitetura centralizada sem comprometer o objetivo principal de E2EE.

O servidor aceita ligações de vários clientes através de sockets TCP. Cada ligação é tratada numa thread própria, permitindo que vários utilizadores estejam ligados em simultâneo. Sobre a ligação TCP é usada uma camada TLS, de modo a proteger a comunicação cliente-servidor contra observação e manipulação na rede.

3.2. Organização do projeto

O projeto está organizado em três áreas principais: `common`, `server` e `client`.

A pasta `common` contém código partilhado entre cliente e servidor. Aqui estão definidas as constantes do protocolo, o formato das mensagens, as funções de serialização, o framing, utilitários criptográficos, validações comuns, suporte TLS e funções auxiliares para permissões de ficheiros.

A pasta `server` contém a implementação do servidor. O ficheiro principal é `server.py`, que aceita ligações, valida pedidos, despacha ações e mantém o estado dos utilizadores online. A gestão persistente de utilizadores é feita em `user_db.py`, enquanto o armazenamento de mensagens é tratado em `message_store.py`.

A pasta `client` contém a aplicação usada por cada utilizador. O cliente implementa a interface de linha de comandos, o envio de pedidos ao servidor, o processamento de respostas, a gestão local de chaves e a cifragem/decifragem das mensagens. O ficheiro `commands.py` traduz os comandos textuais do utilizador para ações do protocolo, enquanto `key_manager.py` gere o material criptográfico local.

Esta separação foi adotada para manter responsabilidades bem definidas. O servidor trata da coordenação e persistência, o cliente trata das operações criptográficas sensíveis, e a pasta `common` concentra os elementos necessários para que ambos usem o mesmo protocolo e as mesmas convenções.

3.3. Protocolo de comunicação

A comunicação entre cliente e servidor usa um protocolo próprio baseado em JSON. Cada mensagem é enviada com um prefixo de tamanho de 4 bytes, seguido do corpo JSON serializado.

Esta decisão foi tomada porque o TCP é um fluxo contínuo de bytes e não preserva, por si só, as fronteiras entre mensagens. O prefixo de tamanho permite ao recetor saber exatamente quantos bytes deve ler para reconstruir uma mensagem completa.

Ao nível aplicacional, o protocolo distingue três tipos de mensagens:

- `request` : pedido enviado pelo cliente ao servidor;
- `response` : resposta do servidor a um pedido;
- `event` : notificação assíncrona enviada pelo servidor.

Cada pedido contém uma ação, um identificador de pedido (`request_id`) e um payload. O `request_id` permite associar respostas aos pedidos correspondentes. As ações incluem operações como registo, login, gestão de contactos, envio de mensagens, consulta da inbox e confirmação de entrega.

O servidor valida a estrutura dos pedidos antes de executar qualquer ação. Esta decisão segue uma abordagem defensiva: o servidor não assume que o cliente envia sempre dados bem formados e rejeita pedidos inválidos ou com campos inesperados.

3.4. Persistência de dados

A persistência é feita através de ficheiros JSON. Esta escolha simplifica a implementação e é suficiente para um protótipo académico, permitindo observar facilmente o estado do sistema durante testes e defesa.

No servidor, o ficheiro `users.json` guarda a informação dos utilizadores, incluindo usernames, contactos e chaves públicas. O ficheiro `messages.json` guarda os envelopes das mensagens, organizados por destinatário, bem como metadados necessários para entrega e estado.

No cliente, são guardadas localmente as chaves privadas, o estado das sessões criptográficas e o material associado à confiança TLS do servidor. Estes dados permanecem fora do servidor porque são necessários para manter a separação entre coordenação central e proteção ponta-a-ponta.

Para reduzir riscos de corrupção, as estruturas persistentes usam locks e escrita atómica sempre que possível. Além disso, os ficheiros sensíveis são protegidos com permissões restritas em sistemas que suportam este mecanismo.

Esta solução tem limitações: a persistência em JSON não é adequada para grande escala, não oferece transações complexas e continua a expor metadados administrativos a quem tenha acesso ao disco do servidor. Ainda assim, para o âmbito académico do projeto, permite manter a arquitetura simples, auditável e coerente com os objetivos definidos.

3.5. Decisões arquiteturais principais

A tabela seguinte resume as principais decisões tomadas durante o desenvolvimento, bem como a respetiva justificação e limitação.

Mecanismo	Decisão tomada	Justificação	Limitação
Arquitetura cliente-servidor	Servidor central coordena clientes	Simplicidade, persistência e suporte a mensagens offline	Dependência do servidor
TLS com TOFU	Canal cliente-servidor protegido	Reduz exposição a ataques de rede	Primeira ligação continua sensível
Challenge-response Ed25519	Login sem password no servidor	Prova posse da chave privada sem a revelar	Depende da chave pública registada
E2EE	Cifragem feita no cliente	Servidor não acede ao conteúdo	Metadados continuam visíveis
X25519 + HKDF	Derivação de chaves simétricas	Separação entre acordo de chave e chave de cifra	Confiança nas chaves públicas vem do servidor
ChaCha20-Poly1305	Cifra autenticada	Confidencialidade e integridade no mesmo mecanismo	Nonces não podem ser reutilizados
Mensagens offline	Servidor guarda envelopes cifrados	Permite comunicação assíncrona	Servidor controla disponibilidade
Persistência JSON	Ficheiros simples	Adequado a protótipo académico	Pouco escalável
Chaves locais	Guardadas no cliente	Servidor não tem chaves privadas	Sem passphrase

Tabela 1: Resumo das principais decisões arquiteturais.

4. Funcionalidades e Fluxos de Comunicação

4.1. Registo e autenticação

O registo de utilizadores associa cada username a material criptográfico próprio. Quando um utilizador executa o comando de registo, o cliente gera localmente um par de chaves Ed25519 e um par de chaves X25519.

As chaves privadas permanecem apenas no cliente. O servidor recebe e armazena apenas as chaves públicas, juntamente com o username. Esta decisão evita o armazenamento de passwords ou segredos de autenticação no servidor.

O login é implementado através de um mecanismo de challenge-response. O cliente inicia o processo enviando o username. O servidor gera um nonce aleatório e temporário, associando-o à ligação atual. O cliente assina esse nonce com a sua chave privada Ed25519 e envia a assinatura ao servidor.

O servidor verifica a assinatura com a chave pública Ed25519 registada para esse utilizador. Se a verificação for bem-sucedida, a ligação passa a estar autenticada. Esta abordagem foi escolhida porque permite autenticar o utilizador sem transmitir passwords e sem exigir que o servidor armazene segredos reutilizáveis. Além disso, como o desafio é aleatório e tem validade limitada, reduz-se o risco de ataques de replay.

4.2. Gestão de contactos e utilizadores online

O sistema permite adicionar e listar contactos. Optámos por não usar a lista de contactos como mecanismo obrigatório de autorização para manter o fluxo de envio simples e centrado na existência de uma identidade criptográfica válida. Esta decisão simplifica o protótipo, mas significa que os contactos funcionam sobretudo como funcionalidade de organização e não como barreira de segurança.

O servidor também mantém uma lista de utilizadores atualmente online. Esta informação é guardada em memória e protegida por locks, uma vez que o servidor pode lidar com várias ligações em simultâneo.

A existência de contactos e presença online melhora a usabilidade do sistema, mas não altera o modelo principal de segurança, que assenta na identidade criptográfica dos utilizadores e na cifragem ponta-a-ponta.

4.3. Envio e receção de mensagens

O envio de mensagens é feito sempre a partir do cliente. Antes de enviar uma mensagem ao servidor, o cliente obtém as chaves públicas do destinatário, prepara o envelope criptográfico, cifra o conteúdo e assina os dados relevantes.

O servidor recebe apenas o envelope cifrado. A sua função é validar a estrutura do pedido, associar o remetente à sessão autenticada, verificar se o destinatário existe e armazenar ou encaminhar a mensagem.

Na receção, o destinatário consulta a inbox e o cliente verifica e tenta decifrar localmente cada mensagem. Se a validação ou decifragem falhar, a mensagem não é confirmada ao servidor.

Esta decisão é importante porque impede que o servidor marque como entregue uma mensagem que o cliente recebeu mas não conseguiu processar. Assim, a entrega técnica depende da validação bem-sucedida no lado do destinatário.

4.4. Mensagens offline e confirmação de entrega

Uma das valorizações implementadas foi o suporte a mensagens offline. Quando o destinatário não está ligado, o servidor guarda o envelope cifrado no ficheiro de mensagens com estado `pending`.

Quando o destinatário consulta a inbox, o servidor devolve as mensagens pendentes. O cliente processa cada mensagem localmente e só envia um acknowledgement (`ack_inbox`) para as mensagens que foram validadas e decifradas com sucesso.

Após receber o ack, o servidor marca essas mensagens como `delivered`. Esta decisão evita que uma mensagem seja considerada entregue apenas por ter sido enviada pelo servidor. A entrega só é confirmada depois de o cliente destinatário a conseguir processar.

4.5. Sessões criptográficas e rekey

Para evitar uma dependência excessiva das chaves de longo prazo, foi implementado um mecanismo de sessões criptográficas por direção.

Cada direção de comunicação mantém um estado próprio, composto por `session_id`, `chain_key`, contador, expiração e limite de mensagens. Quando uma sessão deixa de ser considerada válida, é criada uma nova sessão através de rekey.

Foram também introduzidas reply prekeys, que permitem usar material efémero anunciado pelo outro utilizador em sessões futuras. Esta abordagem melhora parcialmente as propriedades de forward secrecy, pois reduz a reutilização prolongada de material criptográfico associado às chaves estáticas.

Esta opção foi escolhida como compromisso entre segurança e complexidade. Um protocolo Double Ratchet completo exigiria gestão de mensagens fora de ordem, skipped keys, ratchet Diffie-Hellman bidirecional e recuperação pós-compromisso mais complexa. Para o âmbito do projeto, considerámos mais adequado implementar uma melhoria intermédia, mais simples de testar e defender.

Assim, a solução implementada não deve ser apresentada como equivalente ao protocolo usado pelo Signal, mas sim como uma valorização intermédia de forward secrecy adequada ao contexto académico do trabalho.

5. Modelo de Segurança e Gestão de Chaves

5.1. Modelo de ameaça

O modelo de segurança assume que o servidor é honesto mas curioso. Isto significa que o servidor executa corretamente as funcionalidades esperadas, como encaminhamento, armazenamento e gestão de utilizadores, mas não deve conseguir aceder ao conteúdo das mensagens.

Assume-se também a existência de um atacante de rede capaz de observar, modificar, injetar ou repetir mensagens na comunicação cliente-servidor. Por esse motivo, a comunicação com o servidor é protegida por TLS.

O sistema não assume que o disco do cliente ou do servidor esteja protegido contra um atacante com acesso local completo. Essa limitação é relevante sobretudo para as chaves privadas e estado de sessões guardados localmente no cliente.

5.2. Servidor honesto mas curioso

O servidor é necessário para o funcionamento do sistema, mas não deve ser confiado para confidencialidade do conteúdo das mensagens.

Por isso, as mensagens são cifradas no cliente antes de serem enviadas. O servidor apenas armazena envelopes cifrados e metadados necessários para entrega, como remetente, destinatário, timestamp e estado da mensagem.

Esta decisão permite manter uma arquitetura centralizada simples, com suporte a mensagens offline, sem dar ao servidor acesso ao plaintext.

5.3. Atacante de rede e proteção contra MITM

Como se assume a possibilidade de ataques de man-in-the-middle, a comunicação entre cliente e servidor é protegida com TLS. O servidor possui uma identidade TLS persistente e o cliente aplica um modelo TOFU com pinning local.

Na primeira ligação, o cliente aceita e guarda a identidade TLS observada. Nas ligações seguintes, a fingerprint do certificado é comparada com a identidade previamente confiada. Se a identidade mudar inesperadamente, a ligação é rejeitada.

Esta solução não substitui uma PKI completa, porque a primeira ligação continua sensível. Ainda assim, melhora significativamente o modelo inicial, impedindo alterações silenciosas da identidade do servidor após a primeira confiança.

5.4. Identidade dos utilizadores

Cada utilizador é identificado por um username único e por material criptográfico próprio. O cliente gera localmente um par Ed25519, usado para autenticação e assinaturas, e um par X25519, usado para acordo de chaves.

A chave privada nunca é enviada ao servidor. O servidor guarda apenas as chaves públicas associadas ao username. Esta associação é usada no login e na descoberta de chaves públicas entre utilizadores.

5.5. Gestão de chaves no cliente e no servidor

O cliente é responsável por gerar, armazenar e usar as chaves privadas. Estas chaves são guardadas localmente em ficheiros, com permissões restritas quando o sistema operativo o suporta.

O servidor guarda apenas material público: chaves públicas Ed25519 e X25519 dos utilizadores. Estas chaves são disponibilizadas a outros clientes quando é necessário cifrar mensagens para um destinatário.

Esta decisão reduz a quantidade de material sensível armazenado no servidor. No entanto, significa que a confiança na associação entre username e public key depende do servidor.

5.6. Confiança, TOFU e limitações de identidade

A solução usa TOFU e pinning para a identidade TLS do servidor, mas não implementa uma PKI completa para utilizadores. Assim, o servidor continua a funcionar como diretório central de chaves públicas.

Esta abordagem é simples e adequada ao protótipo desenvolvido, mas tem limitações. Um modelo mais forte exigiria certificados de utilizador, uma autoridade de certificação, pinning por contacto ou outro mecanismo de validação independente da associação entre username e chave pública.

6. Primitivas Criptográficas e Garantias

6.1. Ed25519

Ed25519 é usado para assinaturas digitais. No projeto, esta primitiva é usada em dois momentos principais: autenticação do utilizador através de challenge-response e assinatura dos envelopes das mensagens.

A escolha de Ed25519 deve-se ao facto de ser uma primitiva moderna, eficiente e adequada para assinaturas digitais. Permite que o cliente prove posse da chave privada sem a revelar e permite ao destinatário validar a origem dos envelopes recebidos.

6.2. X25519 e HKDF-SHA256

X25519 é usado para acordo de chaves entre utilizadores. A partir da chave privada de um lado e da chave pública do outro, é obtido segredo partilhado.

Esse segredo não é usado diretamente como chave de cifra. Em vez disso, é processado por HKDF-SHA256, que deriva chaves simétricas adequadas ao contexto da mensagem ou da sessão.

Esta separação entre acordo de segredo e derivação de chaves segue uma abordagem mais robusta e modular, permitindo incluir contexto criptográfico e evitar reutilização direta de material bruto.

6.3. ChaCha20-Poly1305

ChaCha20-Poly1305 é usado para cifragem autenticada das mensagens. Esta primitiva oferece confidencialidade e integridade numa única operação, através do modelo AEAD.

A utilização de AEAD é importante porque permite proteger o conteúdo cifrado e, ao mesmo tempo, autenticar dados associados que permanecem públicos, como metadados necessários ao processamento da mensagem.

O nonce usado na cifra tem de ser único para cada chave. Por isso, o cliente gera nonces aleatórios e o sistema evita reutilizar prolongadamente o mesmo material criptográfico através do mecanismo de sessões e rekey.

6.4. TLS

TLS é usado para proteger a comunicação entre cliente e servidor. Esta camada garante confidencialidade e integridade no canal de transporte, reduzindo a exposição a atacantes de rede.

A E2EE protege o conteúdo das mensagens entre utilizadores, enquanto o TLS protege também pedidos ao servidor, respostas, eventos, autenticação e obtenção de chaves públicas. As duas camadas têm papéis diferentes e complementares.

6.5. Garantias de segurança

A confidencialidade do conteúdo das mensagens é garantida pela E2EE. As mensagens são cifradas no cliente remetente e apenas decifradas no cliente destinatário. O servidor não possui as chaves privadas dos utilizadores nem as chaves simétricas derivadas para cifrar as mensagens.

A integridade das mensagens é garantida pela cifra autenticada ChaCha20-Poly1305 e pela assinatura Ed25519 dos envelopes. Qualquer alteração ao ciphertext ou aos dados autenticados relevantes causa falha na validação ou decifragem.

A autenticidade é garantida em dois níveis. O servidor autentica o utilizador através de challenge-response, e os clientes verificam assinaturas Ed25519 nos envelopes das mensagens.

A proteção contra replay é considerada no login através do uso de nonces aleatórios e temporários. No envio de mensagens, os counters e sessões também reduzem reutilizações indesejadas de material criptográfico.

A proteção contra MITM no canal cliente-servidor é fornecida por TLS com TOFU e pinning. Esta proteção é complementar à E2EE, mas não substitui uma PKI completa.

7. Valorizações e Hardening

7.1. Mensagens offline

As mensagens offline foram implementadas como valorização funcional e de segurança. O servidor armazena envelopes cifrados quando o destinatário não está ligado.

Como as mensagens são guardadas já cifradas, o suporte offline não compromete a confidencialidade do conteúdo. O estado **pending** indica que a mensagem ainda não foi confirmada pelo cliente destinatário.

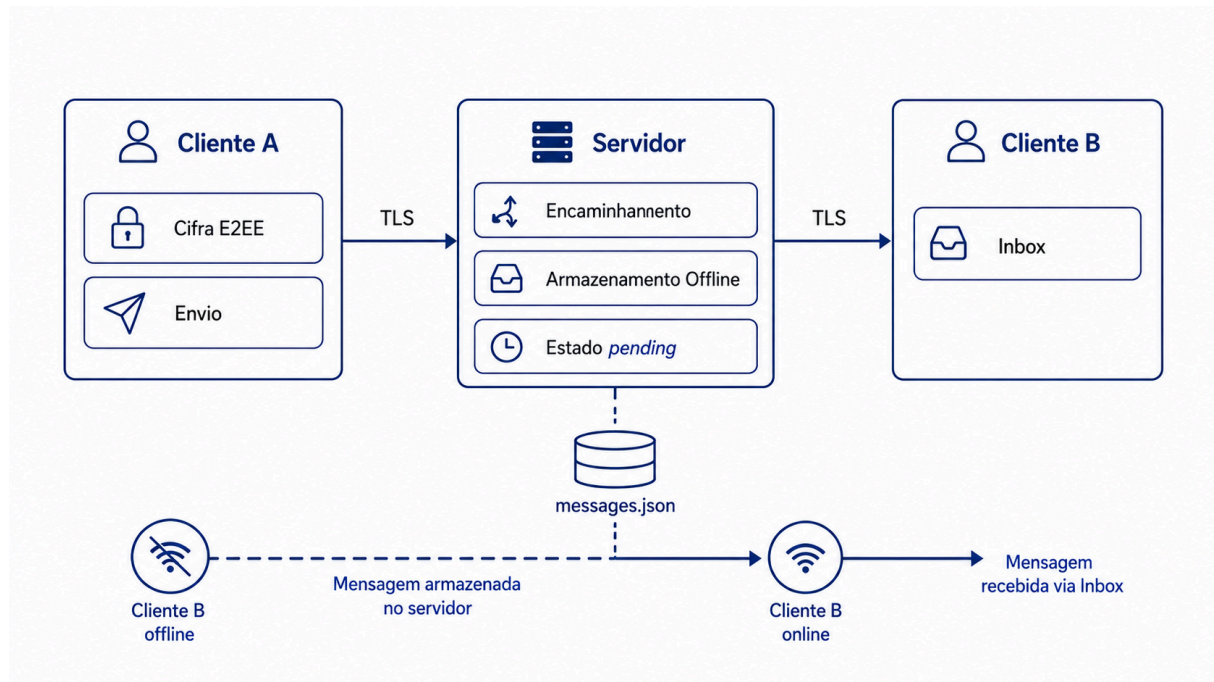


Figura 2: Fluxo de Mensagens Offline.

7.2. Ack de entrega

Foi implementado um mecanismo de acknowledgement para confirmar a entrega técnica das mensagens. O cliente só envia ack depois de conseguir processar, validar e decifrar a mensagem.

Esta decisão evita que o servidor marque uma mensagem como entregue apenas por a ter devolvido na inbox. Se a mensagem estiver corrompida ou não puder ser decifrada, permanece pendente.

O ack representa entrega ao cliente, não leitura humana. Ou seja, uma mensagem marcada como **delivered** foi processada tecnicamente pelo cliente, mas isso não significa que o utilizador a tenha lido conscientemente.

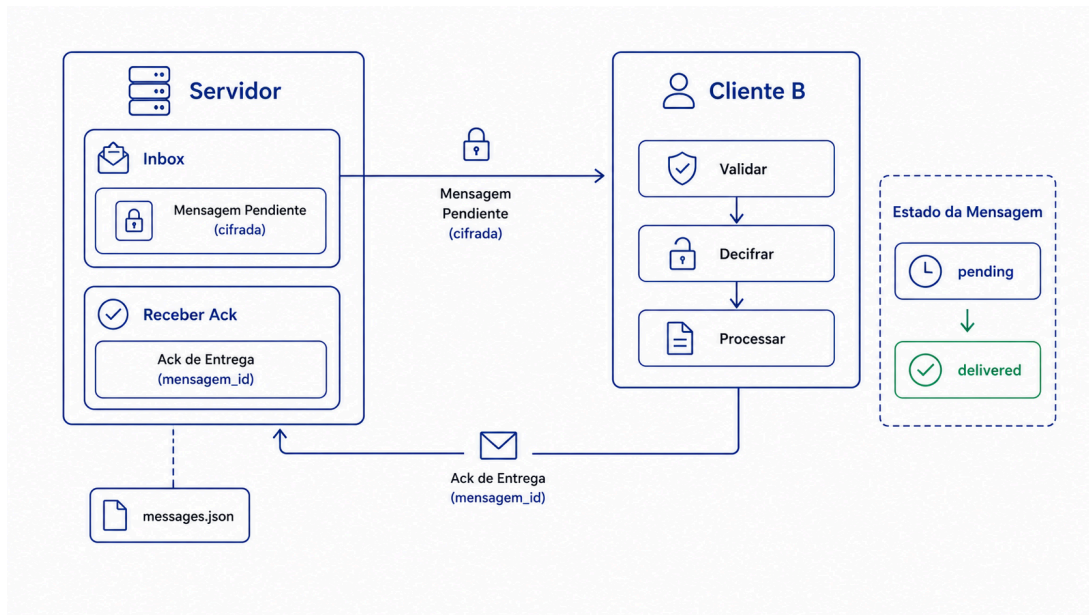


Figura 3: Fluxo de Ack e Inbox.

7.3. Sessões com rekey e reply prekeys

Foi implementada uma valorização intermédia de forward secrecy através de sessões criptográficas por direção.

Cada sessão tem contador, limite de mensagens e expiração. Quando uma sessão deixa de ser considerada segura ou válida, é criada uma nova sessão. As reply prekeys permitem que respostas futuras usem material efêmero previamente anunciado pelo outro utilizador.

Esta abordagem melhora a solução face a uma utilização direta e repetida das chaves estáticas, mas não deve ser confundida com um protocolo double ratchet completo.

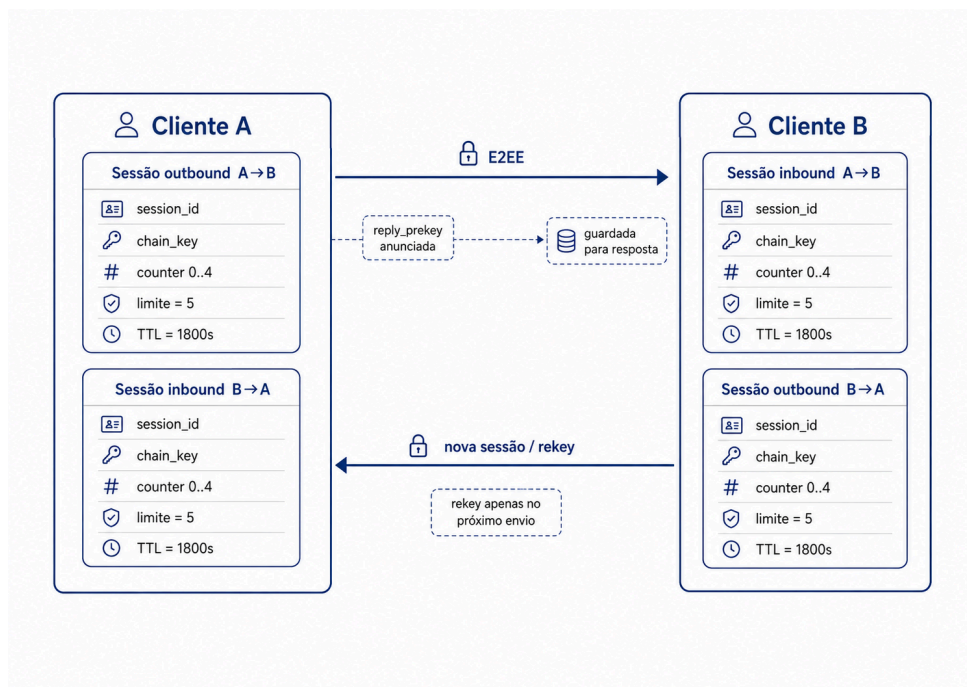


Figura 4: Sessões criptográficas, rekey e reply prekeys

7.4. Validação de inputs e controlo de acesso

Na fase final do projeto, foi realizada uma ronda de hardening. O servidor passou a validar de forma mais rigorosa usernames, request IDs, message IDs, session IDs, metadata, payloads e material criptográfico.

Além disso, as ações sensíveis exigem autenticação e usam sempre o username associado à sessão autenticada como fonte de verdade. O servidor não confia num campo `sender` enviado pelo cliente.

Estas medidas reduzem o risco de erros de lógica, manipulação de pedidos e vulnerabilidades comuns associadas a input malformado.

7.5. Proteção de ficheiros sensíveis

Os ficheiros sensíveis, como chaves privadas, estados de sessão e dados persistentes, são protegidos com permissões restritas quando possível.

Esta proteção é feita em regime de “best effort” ou “quando suportado pelo sistema operativo, uma vez que depende do sistema operativo. Em sistemas Unix-like, são usadas permissões como `0600` para ficheiros sensíveis e `0700` para diretórios privados.

Apesar disso, as chaves privadas locais não são cifradas com passphrase, o que permanece uma limitação importante da solução.

7.6. Smoke tests realizados

Foram realizados smoke tests para validar os principais fluxos do sistema ao longo do desenvolvimento. Na fase de autenticação, foram testados o registo e login de dois utilizadores, adição e listagem de contactos, consulta de utilizadores online, envio de mensagem e consulta da inbox.

Foram também testados casos de erro relevantes, como login de utilizador inexistente e tentativa de login de um utilizador registado mas sem chave privada local. Estes testes ajudaram a confirmar que o novo mecanismo de challenge-response não quebrava as funcionalidades da Etapa A.

Na fase final, foi criado um smoke test para validar o fluxo completo já com TLS, E2EE, mensagens offline, consulta da inbox e ack de entrega. Este teste não substitui uma suite completa de testes, mas permite verificar rapidamente que os principais componentes continuam integrados e funcionais após alterações ao código.

8. Limitações e Trabalho Futuro

8.1. Limitações da solução

Apesar das garantias implementadas, a solução apresenta limitações importantes.

A primeira limitação é a ausência de uma PKI completa para utilizadores. O servidor funciona como diretório central de chaves públicas, pelo que a associação entre username e chave pública depende da confiança no servidor.

O modelo TOFU usado para TLS também tem limitações. A primeira ligação é sensível, porque o cliente ainda não possui uma identidade previamente confiada do servidor.

As chaves privadas locais são guardadas sem passphrase. Embora sejam aplicadas permissões restritas, um atacante com acesso local ao dispositivo do cliente pode comprometer esse material.

O servidor não vê o conteúdo das mensagens, mas continua a observar metadados como remetente, destinatário, timestamp, estado das mensagens e tamanho aproximado dos envelopes cifrados.

A forward secrecy implementada é parcial. Existem sessões, rekey e reply prekeys, mas não existe double ratchet completo nem recuperação pós-compromisso equivalente à de protocolos modernos como o Signal.

Por fim, a solução não suporta mensagens de grupo, modo descentralizado, múltiplas sessões simultâneas por utilizador ou uma base de dados escalável para produção.

8.2. Melhorias futuras

Como trabalho futuro, seria relevante implementar uma PKI para utilizadores, permitindo que a associação entre identidade e chave pública fosse validada através de certificados.

Outra melhoria importante seria evoluir o mecanismo atual de sessões para um protocolo double ratchet completo, com melhores garantias de forward secrecy e recuperação pós-compromisso.

Também seria desejável proteger as chaves privadas locais com uma passphrase ou integrar o armazenamento com mecanismos seguros do sistema operativo.

Do ponto de vista funcional, poderiam ser acrescentadas mensagens de grupo, read receipts reais, limpeza automática de mensagens entregues e suporte a comunicação mais descentralizada.

Por fim, a persistência poderia ser migrada para uma base de dados mais robusta, com melhor suporte a concorrência, auditoria e escalabilidade.

9. Conclusão

Com este projeto, foi desenvolvido um sistema de conversação seguro que aplica, de forma prática, vários conceitos estudados na unidade curricular de Segurança de Sistemas Informáticos.

A solução implementa autenticação sem passwords, comunicação cliente-servidor protegida por TLS, cifragem ponta-a-ponta das mensagens, assinaturas digitais, suporte a mensagens offline, acknowledgements de entrega e uma valorização intermédia de forward secrecy.

As decisões tomadas procuraram equilibrar segurança, simplicidade e adequação ao contexto académico. O servidor mantém um papel central na coordenação do sistema, mas não possui acesso ao conteúdo das mensagens. As operações criptográficas mais sensíveis são realizadas no cliente, preservando o objetivo de E2EE.

Embora existam limitações, nomeadamente a ausência de PKI completa, o uso de TOFU, a proteção limitada das chaves privadas locais e a inexistência de double ratchet completo, a solução apresenta uma base sólida, modular e defensável.

No geral, o trabalho permitiu consolidar conhecimentos sobre criptografia aplicada, autenticação, gestão de chaves, comunicação segura, persistência e segurança de implementação, demonstrando como diferentes mecanismos devem ser combinados para construir um sistema seguro